

COL216 Assignment 2 Report

In this report we implemented and performed the simulation in C++ for the MIPS pipeline execution of commands in a processor.

PART 1 : 5 Stage pipeline with stalling

We have implemented the pipeline by declaring separate structs for each stage in the pipeline - IF, ID, EX, MEM and WB. Each of these structs have their associated attributes which are required for smooth implementation of the pipeline. Control Signals have also been declared as attributes of each the struct declared above.

Additionally we have also declared a state struct which connects the state of the current and the next clock cycle in the pipeline.

We have ensured that WB takes place in the first half of the clock cycle and all other stages function in the second half by executing the while loop in reverse order , which gurantees that WB takes place before all other stages whenever it is active.

Whether a stage will function or not, depends on the nop value set for that stage.

The pipeline is executed using a while loop. Before running the loop , all the attributes of each struct have been intialised. As the loop runs every stage performs its function when the nop is inactive and sets the attributes for the next stage.

Data hazards have been identified and are efficiently handled by stalling the pipeline .

We have assumed that for beq and bne instructions stalls would be introduced in the pipeline until these instructions reach the EX stage and after they pass from the EX stage the next instruction is fetched. Therefore 2 stalls are inserted in the pipeline till these instructions pass from the EX stage.

For jump instruction the address is obtained in the ID stage and then one stall is introduced and the next instruction corresponding to the PC is fetched.

PART 2: With Bypassing

The 5 structs in this implementation are similar to the previous implementation. The data hazards have been handled by analysing the instructions in the various stages, their destination and operand registers. Stalls have been added when necessary.

PART 3 : 7-9 Stages without bypassing

Here total 9 structs have been declared similar to part 1 and part2 .Design decisions are almost similar to the parts above. Assumptions mentioned above regarding the branch and jump instructions remain the same. For all instructions except the lw and sw MEM_1 and MEM_2 are unused. Stalling conditions have been appropriately handled. Program order and first in first out principle is maintained.

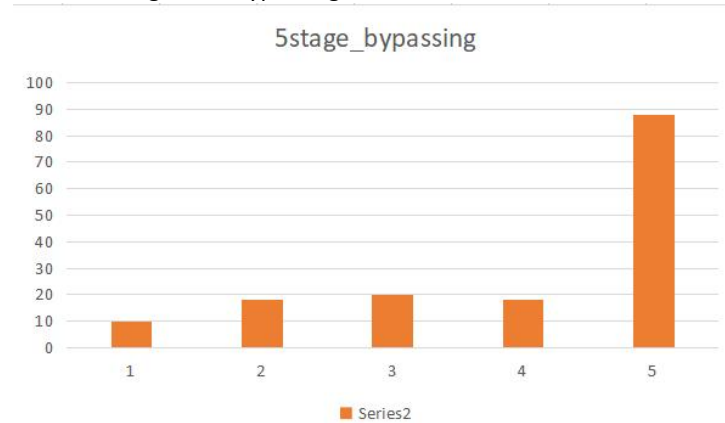
PART 4 : 7-9 stages with bypassing

Implementation remains similar to part 3 . Forwarding conditions have been analysed at appropriate stages and handcdled efficiently.

Plot 1 : 5 Stage without bypassing



Plot 2 : 5 Stage with bypassing

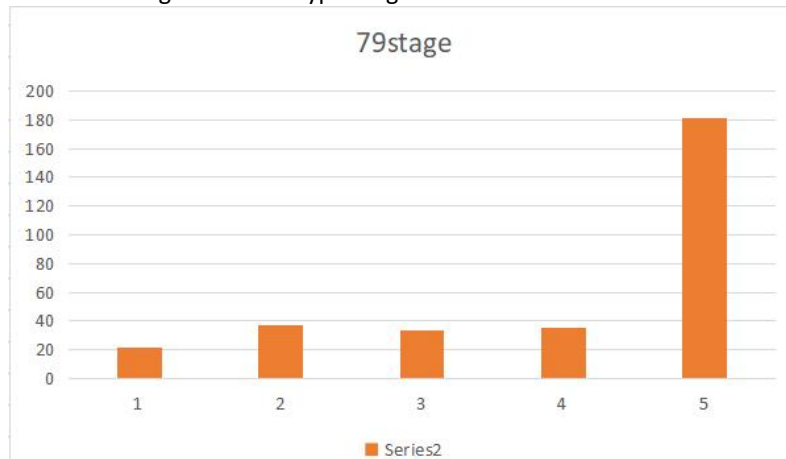


Analysing Plot 1 and Plot 2:

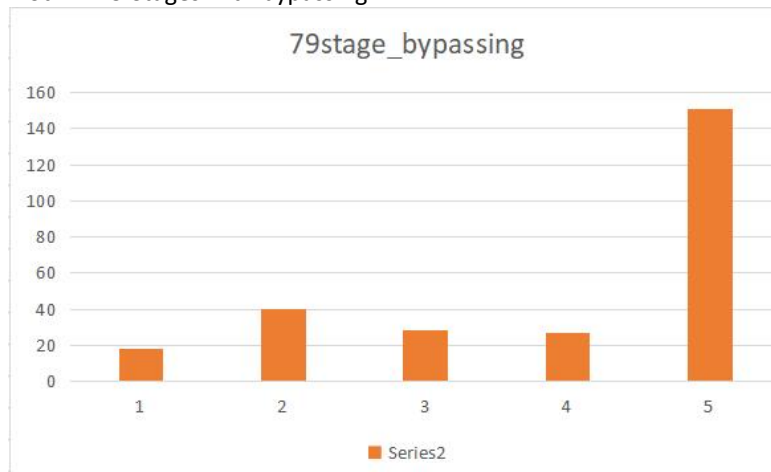
The first plot depicts the no. of cycles required for 5stage.cpp while the second cycle depicts the no. of cycles for the 5stage_bypass .cpp file. From the plots it is clear that the total no. of cycles slightly decreased when forwarding is executed. This is in line with the expectation as stalling delays the execution of the instructions whereas forwarding is faster as instructions as required data is provided earlier.

Testcase	Plot1	Plot2	Improvement
1	16	10	37.5%
2	22	18	18.18%
3	27	20	25.92%
4	27	18	33.33%
sample	89	88	1.1%

Plot 3: 7-9 Stages without bypassing



Plot 4 : 7-9 Stages with bypassing



Analysing Plot 3 and Plot 4 :

The observations for plot 3 and plot 4 are similar to those above . Forwarding results in lesser number of clock cycles.

Testcase	Plot 1	Plot 2	Improvement
1	22	18	18%
2	37	40	-7.5%
3	34	28	17.6%
4	35	27	22.8%
sample	181	161	11.09%

Comparing 5 stage vs 7-9 stage pipeline :

These additional stages allow for more efficient execution of instructions, but they also add more complexity to the pipeline.

In general, a 7-9 stage pipeline may provide better performance than a 5-stage pipeline for certain types of workloads, particularly those that are more complex and require more instructions. As we have implemented using FIFO principle , 7-9 stages take more no. of cycles than 5 stage pipeline.

Part 5 : Branch Predictor

1. Saturating

Value	Accuracy
00	79.0146%
01	83.9416%
10	87.9562%
11	86.6788%

2. BHR

Value	Accuracy
00	73.3577%
01	73.5401%
10	73.9051%
11	74.0876%

3. Combined

Value	Accuracy
00	71.5328%
01	72.2628%
10	87.9562%
11	86.6788%

From the above data it is clear that , the combined predictor may result in better prediction than only BHR and almost same prediction as saturating counters. For the combined predictor we have implemented the "Tournament predictor".

The tournament predictor uses both the predictors: a global predictor (here BHR) and local predictors (here saturating counters). The global predictor makes a prediction based on the history of all branch instructions in the program, while each local predictor makes a prediction based on the history of a specific set of branch instructions.

During program execution, each branch instruction is first predicted by the local predictors, and the prediction of the local predictor with the best history match is chosen. This prediction is then combined with the prediction of the global predictor, which is based on the overall history of all branch instructions. The combined prediction is then used to control the pipeline of the processor, either fetching the instructions from the predicted path or the alternate path.